

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 13 · LECTURE NOTES

Operating It

Observability, failure, and what you can honestly promise.

MODULE OUTCOMES

- Instrument a system with metrics, logs, and traces.
- Explain why tail latency dominates user experience.
- Define SLOs and error budgets and use them to decide.
- Explain cascading failure and the patterns that prevent it.
- State what a distributed system can honestly promise.

1 Observability

Signal	The question it answers	Cost and character
Metrics	Is something wrong, and since when? Aggregate counters, gauges, and histograms.	Cheap, because they are aggregated. Keep cardinality low — a metric labelled by user ID is a time series per user and will bankrupt the monitoring system.
Logs	What exactly happened to this particular request?	Expensive at volume. Sample aggressively, log structured data rather than prose, and always include the trace ID.
Traces	Where did the time go, across all the services this request touched?	The signal that actually finds latency problems in a distributed system, and the one most often missing. See below.
Continuous profiles	Where did the CPU or memory go <i>inside</i> a service?	Now practical to run continuously in production at low overhead, and it closes the gap between a slow span and the reason for it.

TRACE CONTEXT MUST BE PROPAGATED OR TRACING IS USELESS

A distributed trace works because every service passes along the trace identifier it received and attaches its own span to that same trace. The trace is assembled afterwards from independently reported spans that share an ID.

One service that drops the header breaks the trace at that point, and everything downstream of it becomes invisible — which is reliably the part you needed to see, because the problem is usually further in than you expected.

So propagation belongs in the shared framework or middleware, not in each service's own code where it will be forgotten. **And it must cross asynchronous boundaries too** — message queues, background jobs, and scheduled work are exactly where traces are usually lost, because the context does not flow naturally through a queue.

Use W3C Trace Context and OpenTelemetry rather than a vendor format. The standard exists precisely so that one service written in a different language with a different library does not break the chain, which in a polyglot system is not a hypothetical concern.

2 Tail latency

THE TAIL IS THE EXPERIENCE, AND FAN-OUT AMPLIFIES IT

Suppose one service call has a 1% chance of being unusually slow, and a single user request makes 100 such calls. The probability that *none* of them is slow is 0.99^{100} , which is about 37%.

So 63% of requests encounter at least one slow call — and a request that fans out is only as fast as its slowest component, because it cannot return until all of them have.

The p99 of a component becomes the *median* of a fanned-out request. That single sentence is the whole problem, and it explains why a system whose every component looks healthy on average can deliver a consistently poor experience.

So measure p99 and p99.9, and never the mean. The fixes are **hedged requests** (send to a second replica if the first is slow, take whichever answers), **tied requests** (send to two and cancel the loser), and **reducing fan-out** — which is the structural fix. **This is CSCE 735 Module 07's 'report the distribution' and CSCE 650 Module 06's 'the worst frame is the experience', arrived at a third time from a third direction,** which is reasonable evidence it is a general truth about systems people interact with.

SLOS AND ERROR BUDGETS

A service level objective is a target with a measurement attached: 99.9% of requests succeed in under 200 ms, measured at the client, over a rolling 30 days. All four parts matter — the percentile, the threshold, the vantage point, and the window.

The error budget is what remains: 0.1%. Over 30 days that is roughly 43 minutes of permitted failure, and **it is a budget to be spent, not a floor to be defended.**

Budget remaining means ship. Budget exhausted means stop shipping features and spend the time on reliability. The recurring argument between the people who want to ship and the people who want stability becomes arithmetic, settled in advance by a number everyone agreed to — which is the real contribution of the idea, more than the measurement itself.

And 100% is the wrong target. It is unattainable, the last nine costs more than all the preceding ones, and **an error budget that goes unused every month means you are shipping too cautiously** — you bought reliability you did not need with velocity you did.

3 Cascading failure

1. one service slows slightly
2. callers' requests take longer, holding threads and connections
3. callers' pools EXHAUST -- now THEY are slow, for requests unrelated to the original service
4. their callers exhaust, and so on upstream
5. clients time out and RETRY, MULTIPLYING the load
6. the original service, now at several times normal traffic, fails completely
7. it restarts with a cold cache and fails again immediately

PATTERNS THAT BREAK THE CHAIN

TIMEOUTS	bound how long anything is held -- and they must be SHORTER upstream than downstream
BULKHEADS	separate pools per dependency
CIRCUIT BREAKER	after N failures, fail FAST without calling
BACKPRESSURE	reject early rather than queue forever
LOAD SHEDDING	drop low-priority work to protect the rest
JITTERED BACKOFF	so retries do not synchronise into a wave

Retries without backoff and jitter CAUSE outages.

Step 5 is the one that converts a degradation into an outage, and it is entirely self-inflicted. A client library that retries three times immediately turns a service running at its limit into one receiving four times its normal load at exactly the moment it can least handle it. **And unjittered backoff synchronises the retries into waves**, so the service recovers, is immediately flattened by the accumulated wave, and recovers again — the thundering herd, which is a problem the system creates for itself.

TEST THE FAILURES, BECAUSE THEY WILL NOT TEST THEMSELVES

The failure-handling paths are the least-exercised code in the system and among the most important. By definition they do not run in normal operation, so they rot silently — the same argument as CSCE 605 Module 10 made about debug information.

So inject failures deliberately and continuously: kill processes, partition the network, add latency and jitter, fill disks, exhaust connection pools, and slow a dependency without failing it — which is the case that causes cascades and the one that chaos tooling often omits.

Deterministic simulation is better than chaos testing wherever you can manage it. FoundationDB's approach runs the entire system in a single process with a controlled clock and a controlled scheduler, so every source of nondeterminism is under test control and a failing run is *reproducible from its seed*.

That reproducibility is the entire point. A distributed bug you cannot reproduce is a bug you cannot fix — you can only guess, deploy the guess, and wait to see whether it recurs. **Most distributed bugs are not reproducible by default**, which is why building the deterministic harness early (as this course's tooling section asked) pays for itself many times over.

4 Claiming honestly

- **'Linearizable for single-key operations; causal consistency across keys.'** Specific, checkable, and it tells a caller exactly what they may rely on (Module 04).
- **'Tolerates f failures out of $2f+1$ nodes, assuming crash-recovery failures and no Byzantine behaviour.'** **With the failure model stated** (Module 01 §3), because the guarantee means nothing without it.
- **'99.9% of reads complete in under 50 ms, measured at the client, over a rolling 30 days.'** A percentile, a vantage point, and a window — not an average, and not measured on the server where the network is invisible.
- **'Verified with a linearizability checker over 10,000 fault-injected histories, with no violations found.'** **The evidence, and its extent** — which lets the reader judge the confidence rather than being asked to accept it.
- **And what you cannot honestly say: 'highly available', 'never loses data', 'infinitely scalable'. Say the number, the model, and the assumptions. Every course in this program ended on this point** — CSCE 735 about speedups, CSCE 620 about robustness, CSCE 605 about miscompiles, and now this one — **and that is not a coincidence. It is the one habit that transfers to everything.**

WHERE THIS LEAVES YOU

You can reason about partial failure, choose a consistency model deliberately rather than inheriting one, implement consensus and test it adversarially, and build netcode that feels instantaneous while remaining server-authoritative.

And you know several things most practitioners do not: that the impossibility results are considerably narrower than their reputations; that the ambiguity of a single timeout is the root of very nearly everything in the subject; that 'exactly once' does not exist; and that **most systems are distributed some time before they need to be.**

Semester 5 built what the degree had assumed. CSCE 620 supplied the geometry that CSCE 649's collisions and CSCE 647's acceleration structures had taken for granted. **CSCE 605** supplied the compiler behind every shader and every optimisation the program had relied on. **And CSCE 678 supplied the network** — because once the code is correct, fast, and compiled, the only remaining question is how more than one machine runs it together.

Fifteen courses and 195 modules. The thread running through all of them is the same, and it is worth stating one final time: **measure it, state what you established, and never claim more than you tested.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Site Reliability Engineering and the SRE Workbook (free)	SLOs, error budgets, and cascading failure — §2 and §3, from the organisation that formalised them. The 'Addressing Cascading Failures' chapter is the one to read first.
Dean & Barroso — The Tail at Scale (free)	The §2 fan-out arithmetic , with hedged and tied requests. Short, and among the most useful papers in systems.
OpenTelemetry and W3C Trace Context (free)	The §1 standard, and the instrumentation libraries for every major language.
FoundationDB — Testing Distributed Systems with Deterministic Simulation (free)	The §3 approach , which is the strongest testing methodology in this subject and is still not widely adopted.
Kyle Kingsbury — Jepsen (free)	Real distributed databases tested against their own claims , with the results. The best available argument for §4's discipline.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Instrument your Module 06 cluster with metrics, structured logs, and traces.
- 2 **Drop the trace header in one service** and observe what the trace looks like.
- 3 Propagate trace context across a message queue.
- 4 **Measure p50, p99, and p99.9** for your system and report how far apart they are.
- 5 **Simulate the fan-out arithmetic**: 100 calls at 1% slow, and measure the fraction of requests affected. Compare against 0.99^{100} .
- 6 Implement hedged requests and measure the effect on p99.
- 7 Define an SLO for your project and compute the monthly error budget.
- 8 **Reproduce a cascading failure** with no timeouts and immediate retries. Then add timeouts, circuit breakers, and jittered backoff, and show it no longer cascades.
- 9 **Build a deterministic simulation harness** with a controlled clock and scheduler, and reproduce a failure from its seed.
- 10 **Project 2 is now due**. Submit the netcode implementation, the measurements at each latency and loss setting, the reconciliation plot, the demonstrated and defeated cheat, and the honest statement of where it stops working.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name three observability signals and the question each answers.
- 2 Why does trace context propagation decide whether tracing works?
- 3 Explain the fan-out arithmetic and why a component's p99 becomes a request's median.
- 4 Define an SLO and an error budget, and say what an unused budget means.
- 5 Give the seven steps of a cascading failure.
- 6 Name six patterns that break the chain.
- 7 Why do retries cause outages?
- 8 Why is deterministic simulation better than chaos testing?
- 9 Give four honest claims and three things you cannot say.